



INSTITUTO POLITECNICO NACIONAL

ESCUELA SUPERIOR DE COMPUTO

Ingeniería en Inteligencia Artificial

ACADEMIA: Sistemas Digitales

MATERIA: Computo Paralelo

GRUPO: 6BM1

ACTIVIDAD:

**“Simulación Monte Carlo Masiva en la estimación de la
constante matemática π ”**

PRESENTA:

Herrera Monroy Abraham Andre

Marcelino López Jessica

Lobato Ramírez Ulises Uriel

BOLETA: 2024630788

2024630374

2024630651

PLAN: 2020

FECHA: 14 sept 2026

Profesor de Materia:

Rodrigo Ojeda Santillan

1. Planteamiento del Problema

Nuestra propuesta de proyecto consiste en estimar el valor de la constante matemática Pi mediante una simulación de Monte Carlo masiva. Para entender la lógica detrás, imaginemos que tenemos un tablero de dardos cuadrado de 2x2 metros y, justo en el centro, dibujamos un círculo de radio 1 que toca los cuatro bordes. Si lanzo muchísimos dardos de forma completamente al azar, algunos caerán dentro del círculo y otros en las esquinas. Como el área del cuadrado es 4 y la del círculo es π , la proporción geométrica entre ambas nos dice que la probabilidad de que un dardo caiga dentro es exactamente $\pi/4$. Por lo tanto, si logramos simular estos lanzamientos, calcularemos la proporción de aciertos y al multiplicar por 4, obtendremos una aproximación al número Pi.

2. Datos

Para llevar esto a la computadora, los "dardos" serán la generación de N coordenadas aleatorias (x, y) sobre un plano. La computadora evaluará matemáticamente la posición de cada dardo usando el Teorema de Pitágoras con la ecuación $x^2 + y^2 \leq 1$. Si la suma de los cuadrados de las coordenadas es menor o igual a 1, significa que el dardo cayó a una distancia permitida y está dentro del círculo; si es mayor a 1, cayó fuera. El dato principal de nuestro problema es la magnitud de N : a medida que la cantidad de puntos generados tiende al infinito, nuestra aproximación de Pi será mucho más precisa.

3. ¿Por qué es paralelizable?

Si lanzamos pocos puntos, el valor de Pi será sumamente impreciso; necesitamos generar millones o billones de coordenadas. Evaluar esto punto por punto tomaría demasiado tiempo. Sin embargo, este problema es altamente paralelizable porque el cálculo de cada "dardo" es un evento completamente independiente; evaluar la coordenada número 100 no depende en absoluto del resultado de la coordenada número 99. En lugar de procesar los datos de manera secuencial, podemos dividir la carga de trabajo masiva para que múltiples hilos de procesamiento evalúen distintos bloques de coordenadas exactamente al mismo tiempo.

4. Versiones

Versión 1: Secuencial -> Línea base y perfilado

Desarrollar un programa en lenguaje C que se ejecute en un único núcleo y un solo hilo de procesamiento. El algoritmo va a consistir en un ciclo iterativo masivo que generará N partes de coordenadas (x, y) de un punto flotante, evaluándose en la ecuación propuesta. Esto con el objetivo de capturar el tiempo secuencial absoluto.

Versión 2: Memoria compartida (OpenMP)

Para este proyecto se va a utilizar directivas de compilador OpenMP en C. El volumen de iteraciones N se fragmentará y se va a distribuir entre 12 hilos lógicos del procesador local, operando todos sobre el mismo espacio de memoria RAM.

Versión 3: Aceleración masiva por GPU (CUDA)

Para escalar la simulación Monte Carlo, se va a desarrollar un Kernel en CUDA. En lugar de usar muchos hilos en el procesador principal, la carga computacional se va a distribuir en núcleos simultáneos en bloques dentro de la tarjeta gráfica (GPU).